Name:Khushi Singh
Class:D20C Roll no.34

Experiment No.8

Aim: To configure a cryptocurrency wallet (MetaMask) and integrate it with a Decentralized Application (DApp) designed for a secure, traceable pharmaceutical / reagent supply chain on the blockchain (BioToken on the Polygon Amoy test network).

Theory:

1. MetaMask

MetaMask is a browser-extension (and mobile) cryptocurrency wallet that stores the user's private keys and lets the browser talk to EVM blockchains. It signs every transaction or message on the user's behalf only after the user approves it in a pop-up, so websites and DApps never see the private key. It can connect to Ethereum mainnet, test networks and any custom RPC network such as Polygon Amoy.

2. Test Network (Testnet) – Polygon Amoy

A testnet is a copy of a blockchain that behaves like the main network but uses coins with no real-world value, obtained free from a faucet. Polygon Amoy (chain ID 80002, native token POL) is Polygon PoS's public testnet. Every Amoy transaction can be inspected on the block explorer *amoy.polygonscan.com*. Amoy enforces a minimum priority fee (tip) of 25 gwei per transaction.

3. Remix IDE

Remix is a browser-based IDE for writing, compiling, deploying and debugging Solidity contracts. Selecting the environment **Browser Extension → MetaMask** (formerly "Injected Provider – MetaMask") in the Deploy & Run Transactions tab makes Remix send every transaction through MetaMask, so the account and network selected in MetaMask are used.

4. DApp, ethers.js, Privy and Account Abstraction

A DApp has a web front end that reads from and writes to a smart contract instead of a central server. The BioToken front end (React + Vite) reads the contract with *ethers.js / viem* and connects wallets through **Privy**, whose login dialog offers "Continue with a wallet → MetaMask". After the user approves the connection, MetaMask becomes the owner (signer) of an **ERC-4337 smart account**. State-changing calls are packed into a *UserOperation*, which the user signs in MetaMask. The Pimlico bundler submits it to the EntryPoint contract and the Pimlico paymaster sponsors the gas, so the user does not pay POL. Read-only (view) calls are free and need no pop-up.

5. Smart contract used: BioToken

BioToken is an ERC-721 contract (OpenZeppelin ERC721 + AccessControl) in which each NFT represents one reagent / drug batch. It moves through the lifecycle **MINTED → IN_TRANSIT → RECEIVED → VERIFIED → CONSUMED**. Only a Poseidon commitment to the batch's Gold-Standard HPLC fingerprint is stored on-chain, so proprietary data stays private. The main functions are:

- **mintToken(batchId, reagentSmiles, expiry, fingerprintCommitment, tolerance, marketplaceSig)**: creates a batch NFT; caller needs MANUFACTURER_ROLE.
- **transferCustody(tokenId, lab)**: ships the batch to an address holding LAB_ROLE (MINTED → IN_TRANSIT).
- **confirmReceipt(tokenId)**: the receiving lab confirms arrival (IN_TRANSIT → RECEIVED).
- **verifyProof(tokenId, a, b, c, aiAttestation)**: Groth16 ZK proof that the lab's HPLC peaks match the commitment, plus a GENUINE verdict signed by the AI oracle (RECEIVED → VERIFIED).
- **consumeToken(tokenId)**: marks a verified batch as used; this is the terminal state.
- **getTokenData(tokenId)**, **totalMinted()**, **aiVerdictDigest(tokenId)**: free view calls. Invalid transitions revert with `InvalidStatusTransition`.

Tasks Performed:

Part A: Set up MetaMask

MetaMask was installed in Chrome and an existing wallet was used. The Polygon Amoy network (chain ID 80002, symbol POL) was selected and its RPC URL was set to `https://polygon-amoy-bor-rpc.publicnode.com` (see "Problems faced"). Account 1 (`0xbb6E66Fb872cFd173d12a402b11316aab72189B4`) held about 97 POL of test coins. The Secret Recovery Phrase and private key were never shared or shown.

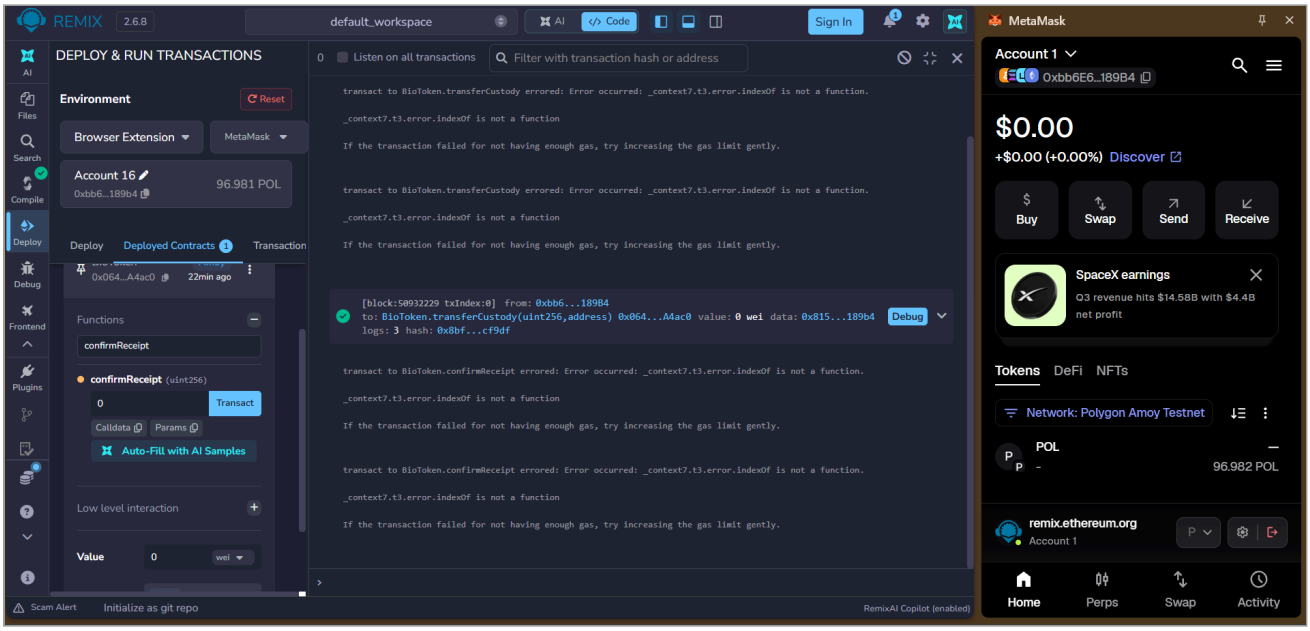


Fig. 1: MetaMask wallet (Account 1) connected to Polygon Amoy Testnet, beside Remix IDE

Part B: Create, compile and deploy the smart contract

BioToken.sol was flattened together with its OpenZeppelin v5.6.1 imports (`npx hardhat flatten`) and pasted into Remix as *BioToken.sol*. In the Solidity Compiler tab the compiler was set to **0.8.28**, with EVM version **cancun** and optimization on (200 runs), the same settings as the project's *hardhat.config.js*. A green tick on the compiler icon confirmed successful compilation.

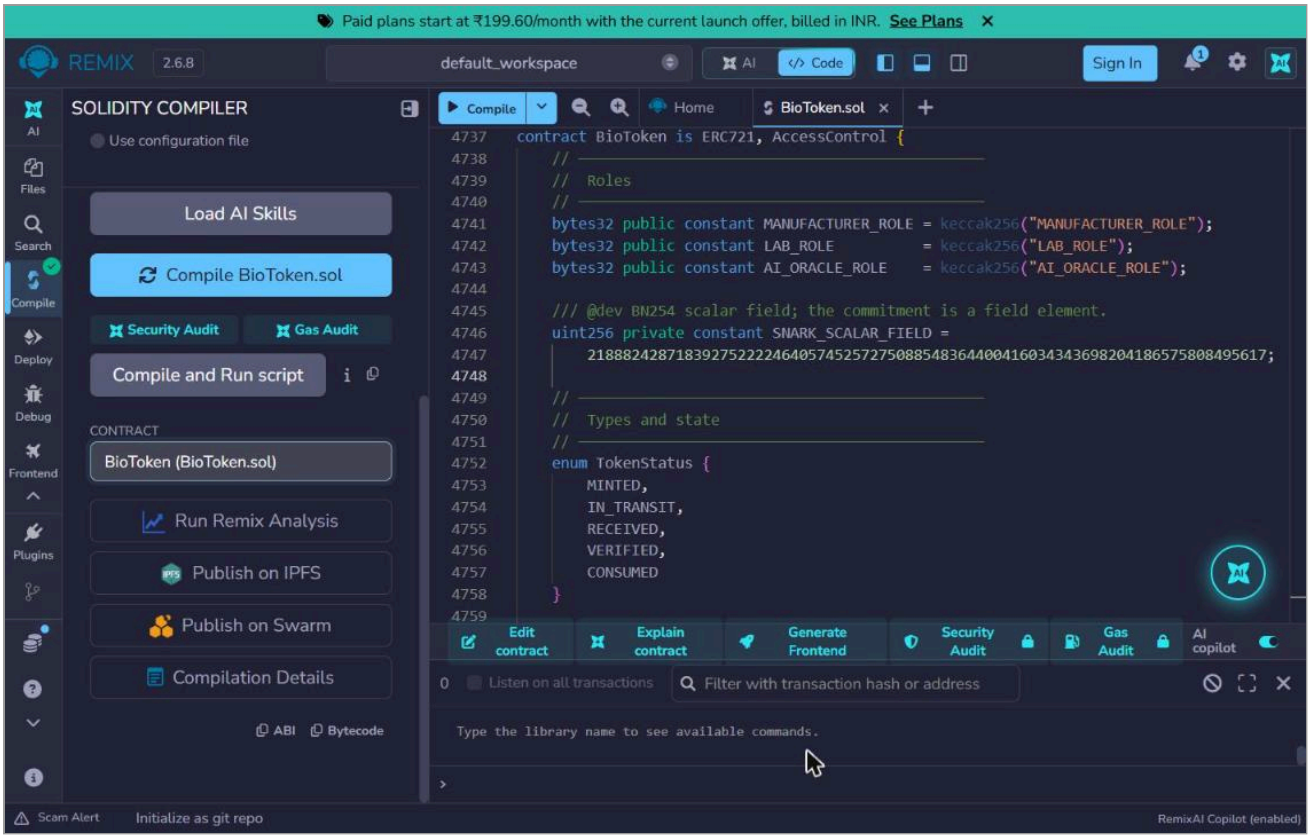


Fig. 2: Successful compilation of BioToken.sol (solc 0.8.28, cancun, optimizer 200)

In Deploy & Run, the environment was set to **Browser Extension → MetaMask**. Remix showed Account 1 with 97.06 POL and the badge "Amoy (80002)". The contract **BioToken** was selected and the constructor arguments were entered: *verifierAddress* = the deployed Groth16 HplcVerifier, *aiOracle* = the AI-oracle signer address. Clicking **Deploy** opened a MetaMask "Deploy a contract" request, which was confirmed. After mining, the contract appeared under "Deployed Contracts".

Item	Value
Contract address	`0x0641266bbc0a7102669388a007930E6c25fA4ac0`
Deployment tx hash	`0x87d346239255ad888729446d2c3b76bf14f75d8d1dfaf81c1f1869881c8f9455`
Block / gas used / fee	50931306 / 2,345,350 gas / 0.06825 POL (gas price 29.1 gwei)
Deployer	`0xbb6E66Fb872cFd173d12a402b11316aab72189B4 (MetaMask Account 1)`
Constructor: verifierAddress	`0xA44699310aE4599cF43C1dBe68B227f3cb4E16DA`
Constructor: aiOracle	`0x5653dc3ef59c4E3d854e0fEB94529A335460c68e`

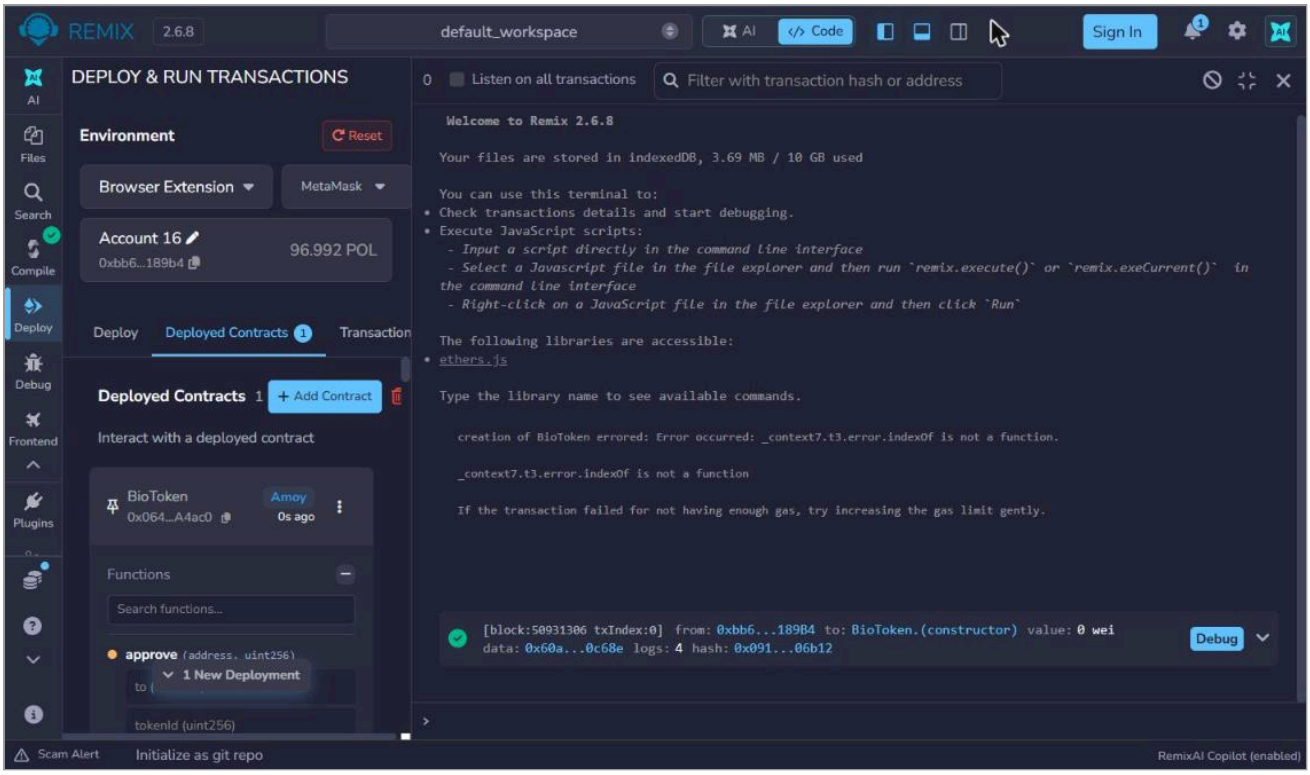


Fig. 3: Deployed BioToken (0x064…A4ac0) on Amoy in Remix, with the successful constructor transaction in block 50931306

Part C: Check the transaction on the block explorer

The deployment transaction hash was searched on *amoy.polygonscan.com*. The explorer shows status **Success**, block 50931306, the sender (Account 1), the created contract address, value 0 POL, the transaction fee 0.06824968514775705 POL and a gas price of 29.1 gwei.

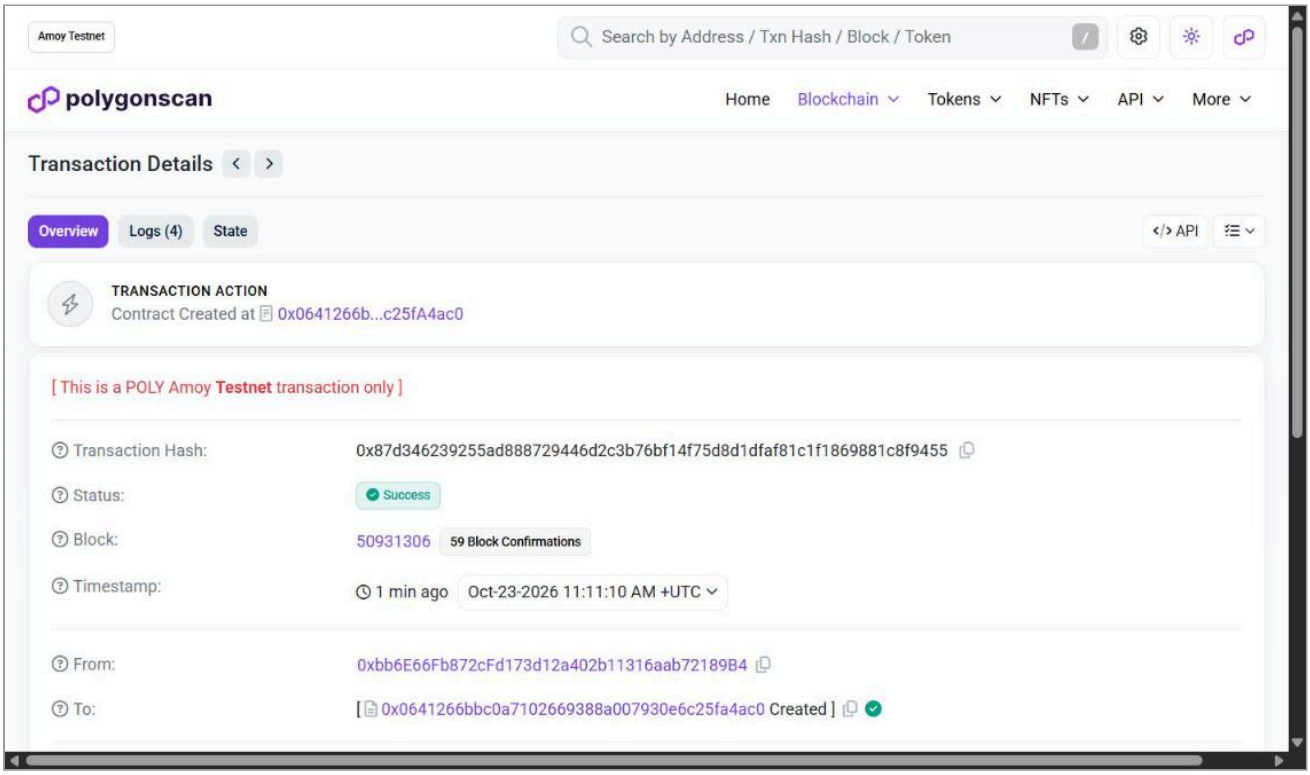


Fig. 4: Deployment transaction on PolygonScan: Contract Created, status Success, block 50931306

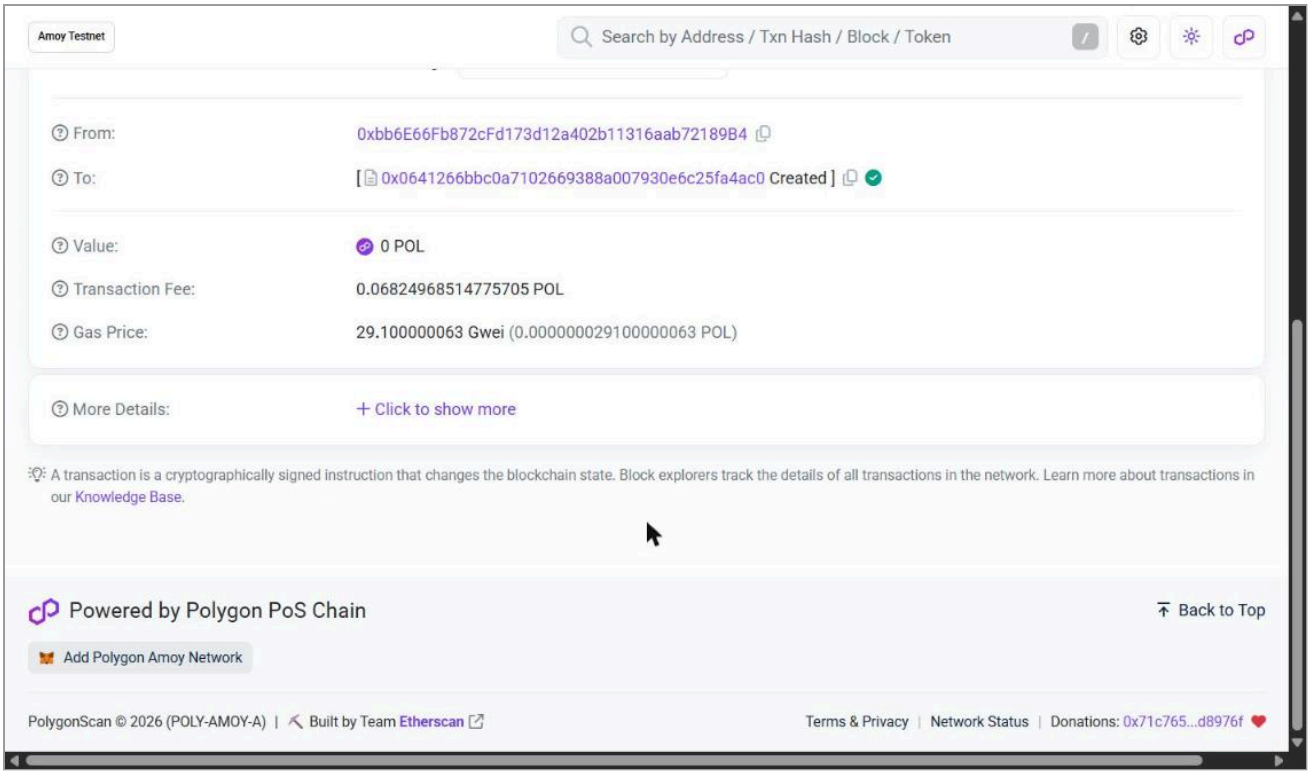


Fig. 5: Deployment transaction details: created contract, value, transaction fee and gas price

Part D: Interact with the smart contract in Remix

Using the deployed-contract panel in Remix, the following calls were made. Each state-changing call opened a MetaMask confirmation. Each view call returned instantly with no gas and no pop-up.

1. **grantRole(LAB_ROLE, Account 1)**: lets the same account act as the receiving laboratory (LAB_ROLE = keccak256("LAB_ROLE") = `0x373f…b2ef`).
2. **mintToken("BT-EXP8-001", "CN1C=NC2=C1C(=O)N(C(=O)N2C)C" (caffeine), 1830297600 (1 Jan 2028), commitment 0x130b…55cb, tolerance 5, 0x)**: Token #0 created.
3. **getTokenData(0)** (free): returned the batch data with status **0 = MINTED**.
4. **transferCustody(0, Account 1)**: the batch was shipped to the lab (MINTED → IN_TRANSIT).
5. **confirmReceipt(0)**: the lab confirmed receipt; **getTokenData(0)** then returned status **2 = RECEIVED**.
6. **Negative test: transferCustody(0, …)** was called again. Gas estimation failed with the custom error `InvalidStatusTransition(tokenId 0, current 2 = RECEIVED, expected 0 = MINTED)`, so the transaction was cancelled and never reached the chain.

Function (Remix → MetaMask)	Block	Gas used	Fee (POL)	Transaction hash
constructor (deploy)	50931306	2,345,350	0.068250	`0x87d346239255ad888729446d2c3b76bf14f75d8d1dfaf81c1f1869881c8f9455`
grantRole(LAB_ROLE, Account 1)	50931458	61,571	0.001792	`0x1d04b94187e44002bca98109336e2bf4188278762afc12c7ef52cd7c1759c794`
mintToken("BT-EXP8-001", …)	50931706	242,530	0.007058	`0x39a8c64bf88d451e3ca80027396e202a6ed36cb307c49a4ac132eda9eb44e4ea`

transferCustody(0, Account 1)	50932229	68,780	0.002001	`0xdc90e6ddceff96e7b470a92b707700f3ce17d30036156da18974b12400e5b4a5`
confirmReceipt(0)	50933008	42,389	0.001272	`0xcf742fccd3a269ae2bd46c8178005f379e1787ebc65072e74cbd3fc9318a9586`

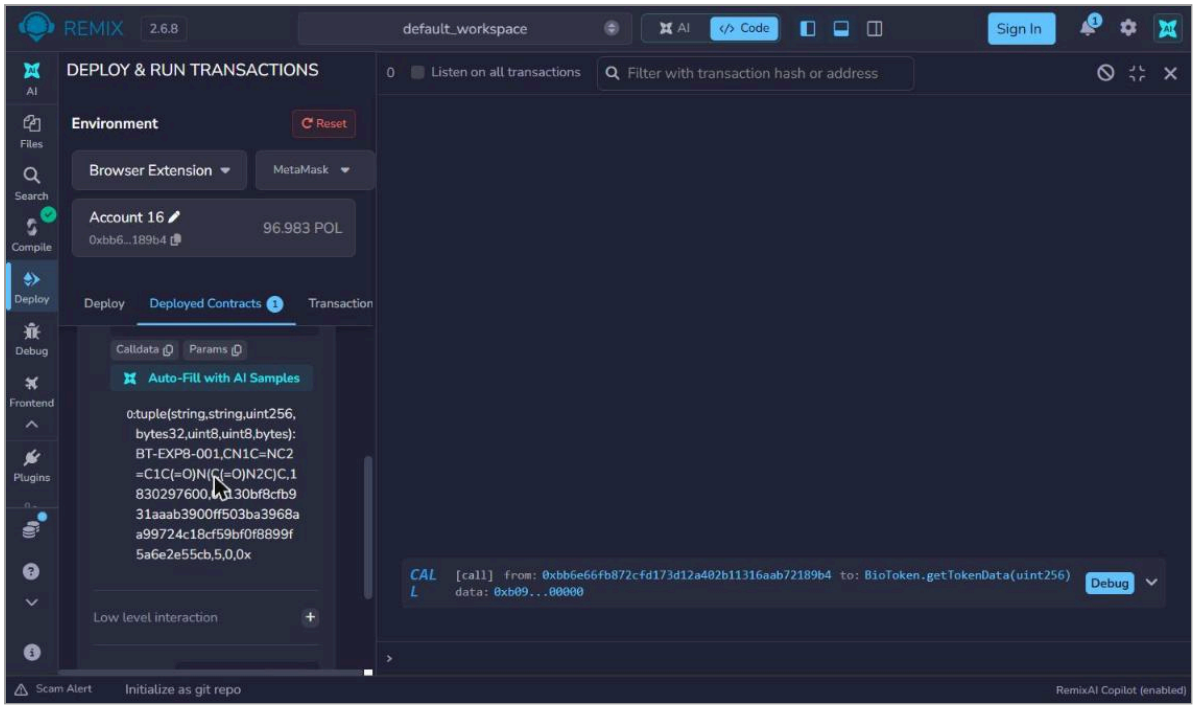


Fig. 6: getTokenData(0) after minting: BT-EXP8-001, caffeine SMILES, expiry, commitment, tolerance 5, status 0 (MINTED)

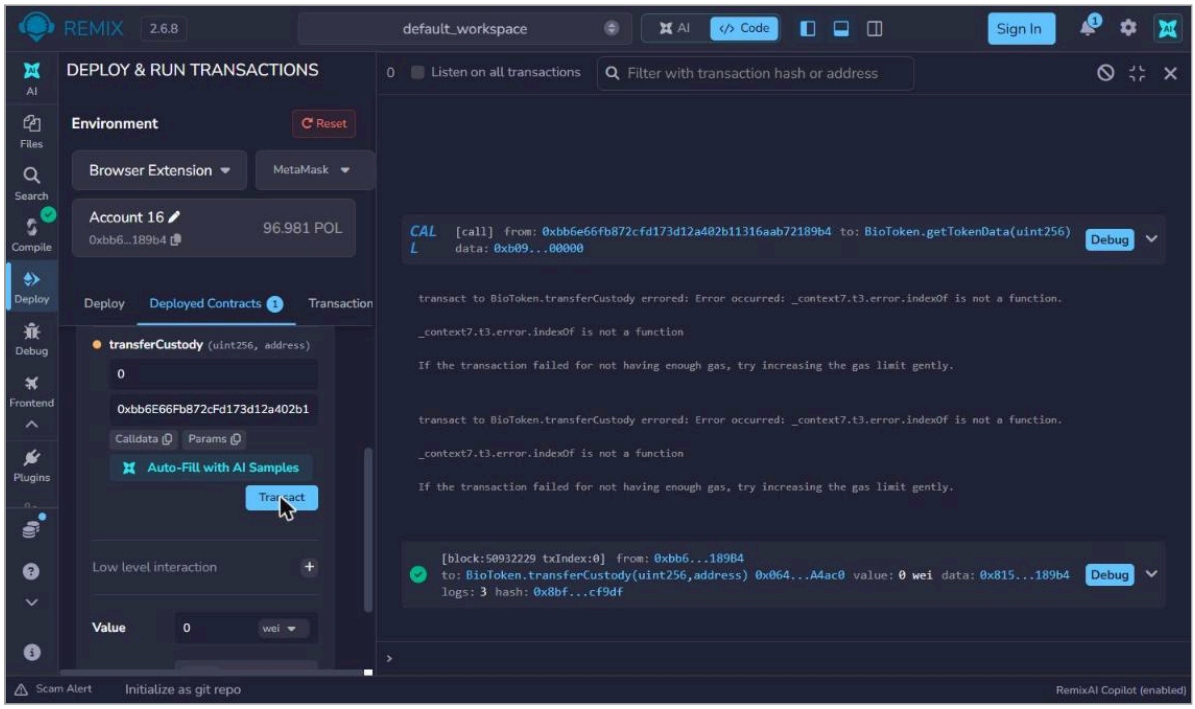


Fig. 7: transferCustody(0, lab) mined in block 50932229 (the earlier "errored" lines are attempts rejected by the network for a too-low tip, see "Problems faced")

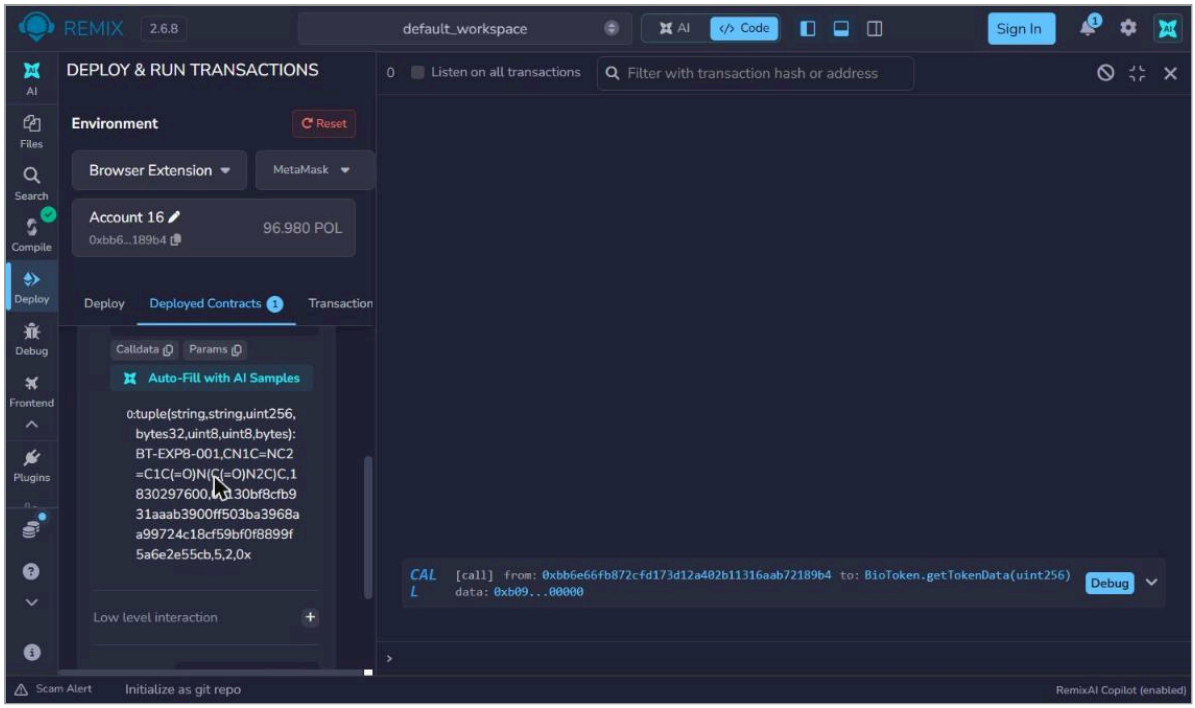


Fig. 8: getTokenData(0) after confirmReceipt: status 2 (RECEIVED)

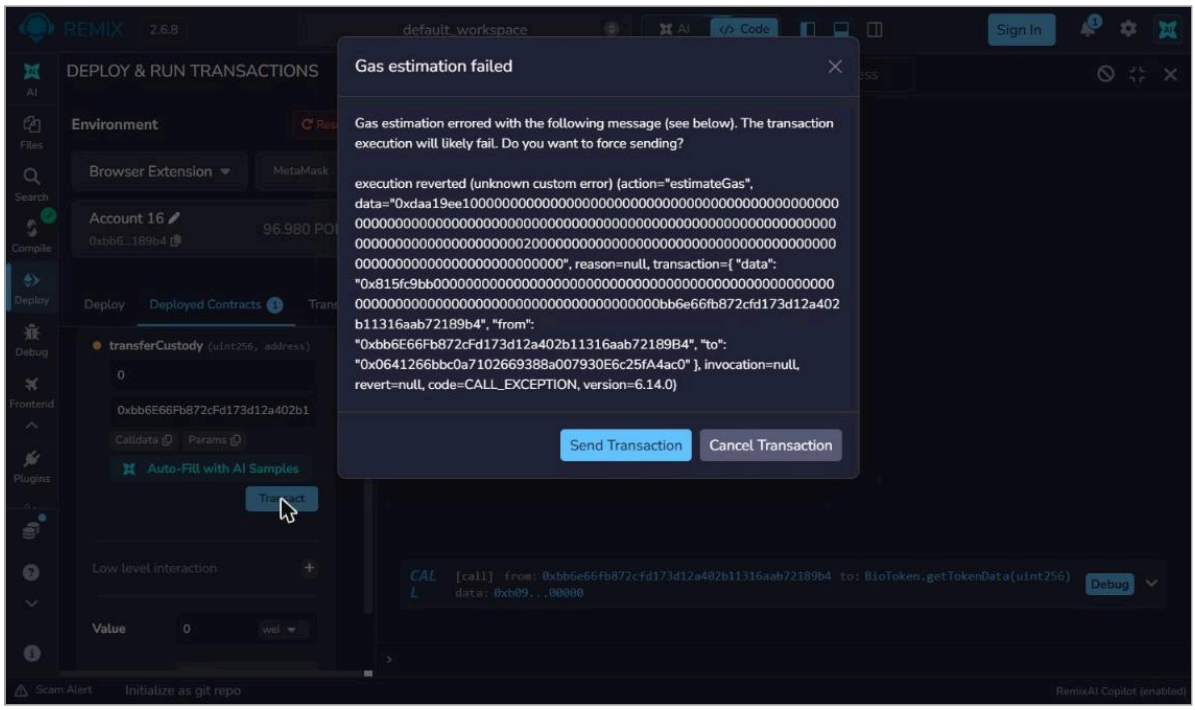


Fig. 9: Negative test: transferCustody on a RECEIVED token reverts with InvalidStatusTransition (selector 0xdaa19ee1), so the transaction is not sent

Part E: Integrate the MetaMask wallet with the DApp

The same deployed contract was then used from the project's own web front end instead of Remix. The steps were:

1. The AI / role service was started with `uvicorn api:app --port 8000` and the front end with `npx vite --port 5173`. Both were pointed at the new contract through the environment variable `VITE_CONTRACT_ADDRESS / CONTRACT_ADDRESS = 0x0641…4ac0`. The service's readiness check confirmed chain 80002, the contract code, AI_ORACLE_ROLE and the admin role.

2. **http://localhost:5173** was opened in the Chrome browser that has MetaMask installed (Fig. 10).
3. **Connect Wallet → Continue with a wallet → MetaMask** was chosen in the Privy dialog. MetaMask asked for permission to connect to the site; it was approved for Account 1 (Figs. 11–12).
4. On the role screen **Manufacturer** was chosen and the registration message was signed in MetaMask. The back end verified the signature and granted **MANUFACTURER_ROLE** on the contract to the user's ERC-4337 smart account `0x133a1e0bbEaFc357bc0950D038cA3b3478a80217` (Fig. 13). The navbar then showed the MFR badge and the connected address `0xbb6E…`, and the dashboard listed the on-chain batches, including Token #0 from Part D (Fig. 14).
5. **Mint a Batch:** the Gold-Standard HPLC file *BATCH-CAF-001_gold_standard.csv* was uploaded and passed the consensus check against 2/2 baseline suppliers. The batch ID *BATCH-CAF-001*, caffeine SMILES and tolerance 10 were entered, and **Commit to Polygon** was clicked. The page computed the Poseidon commitment in the browser and MetaMask asked the user to sign the UserOperation (Figs. 15–16).
6. The DApp showed **"Batch minted · Token #1"** with commitment `0x0cc5beae…8678` (Fig. 17). On PolygonScan the transaction appears as an **Account Abstraction Bundle**: the Pimlico bundler sent it to the EntryPoint and the paymaster paid the gas (Fig. 18). The dashboard now shows both tokens (Fig. 19).

DApp mint (via MetaMask-owned smart account)	**Value**
Token / batch	Token #1, BATCH-CAF-001, status MINTED, expiry 23 Oct 2027, tolerance 10
Transaction hash	`0xcc2fa5bd18a5501d648bf20b09432135b84d4930c81c9c8549876a656b86afce`
Block / gas used	50934231 / 502,223 gas (sponsored by the Pimlico paymaster, 0 POL paid by the user)
Token owner (smart account)	`0x133a1e0bbEaFc357bc0950D038cA3b3478a80217 (owner/signer: MetaMask Account 1)`
Submitted by / to	`bundler 0x433702948d7be4201A2bCEe6C349ad807C547A5D → EntryPoint v0.7 0x0000000071727De22E5E9d8BAf0edAc6f37da032`

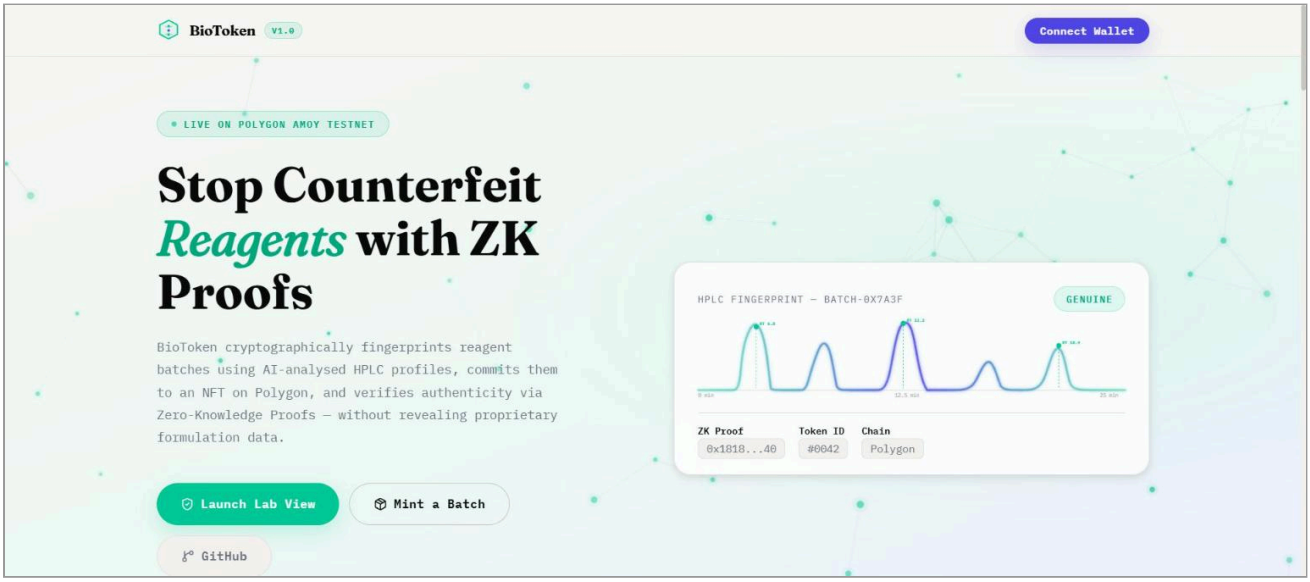


Fig. 10: BioToken DApp loaded at localhost:5173 ("Live on Polygon Amoy Testnet")

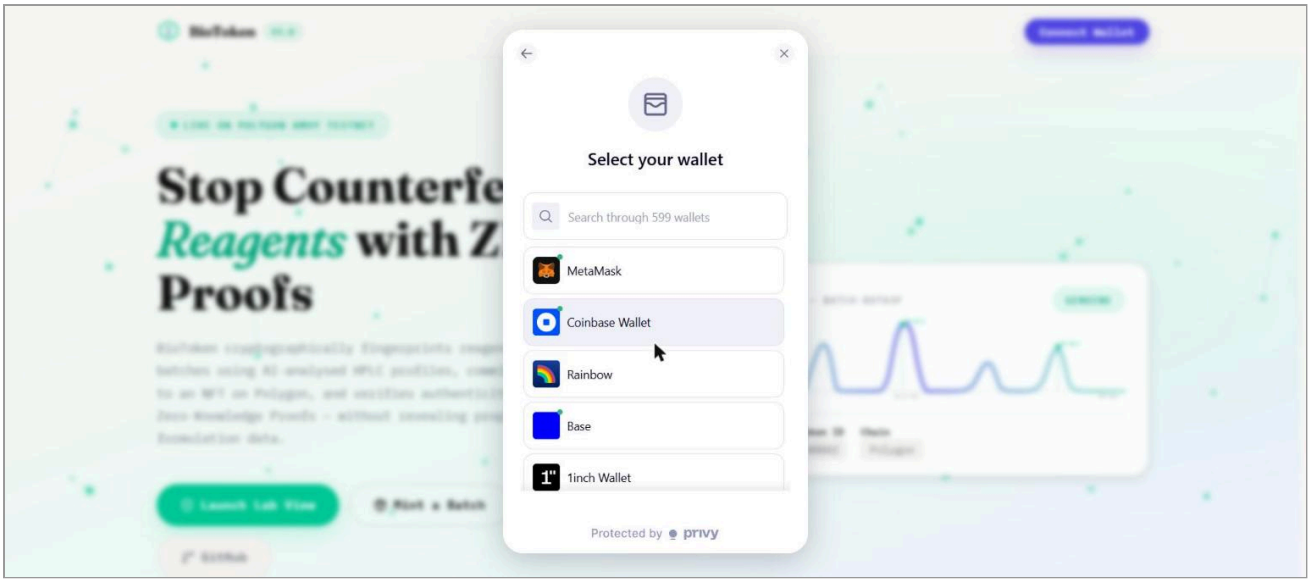


Fig. 11: Privy wallet selection: MetaMask chosen ("Continue with a wallet")

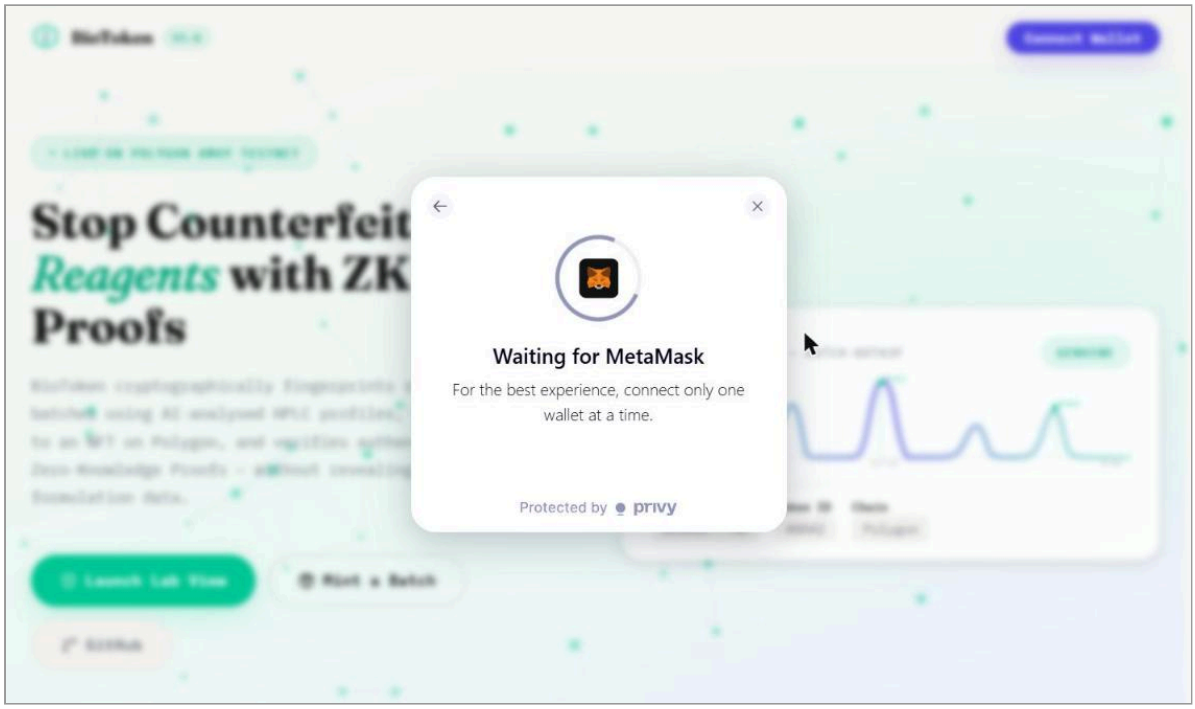


Fig. 12: The DApp waits while the user approves the connection request in MetaMask



Fig. 13: Role selection after connecting MetaMask (Manufacturer chosen; role granted on-chain)

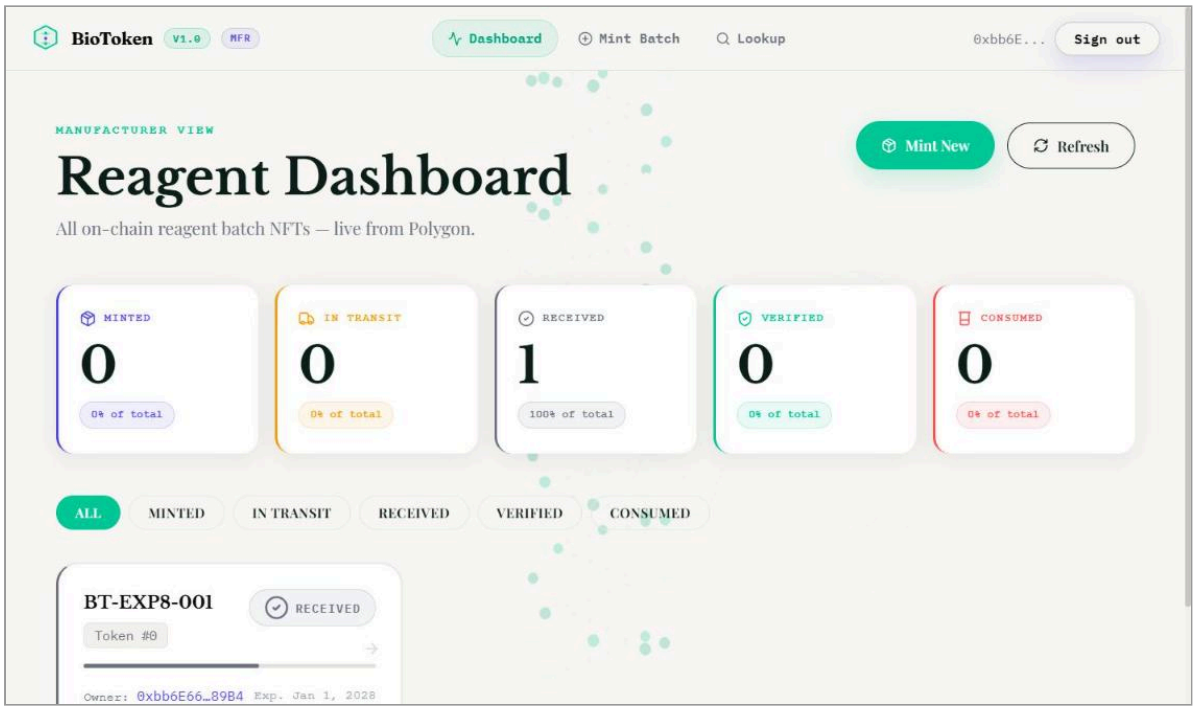


Fig. 14: Manufacturer dashboard with the connected MetaMask account (0xbb6E…) showing live on-chain data (Token #0 RECEIVED)

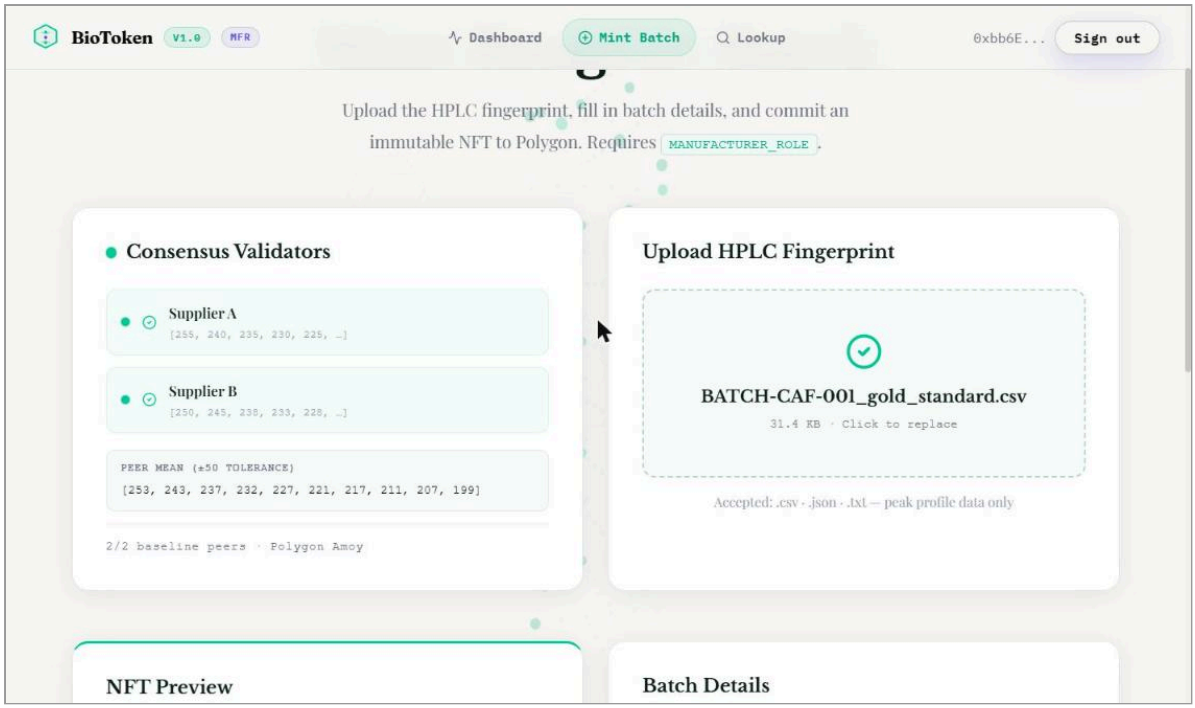


Fig. 15: Mint page: Gold-Standard HPLC file uploaded, consensus validators passed

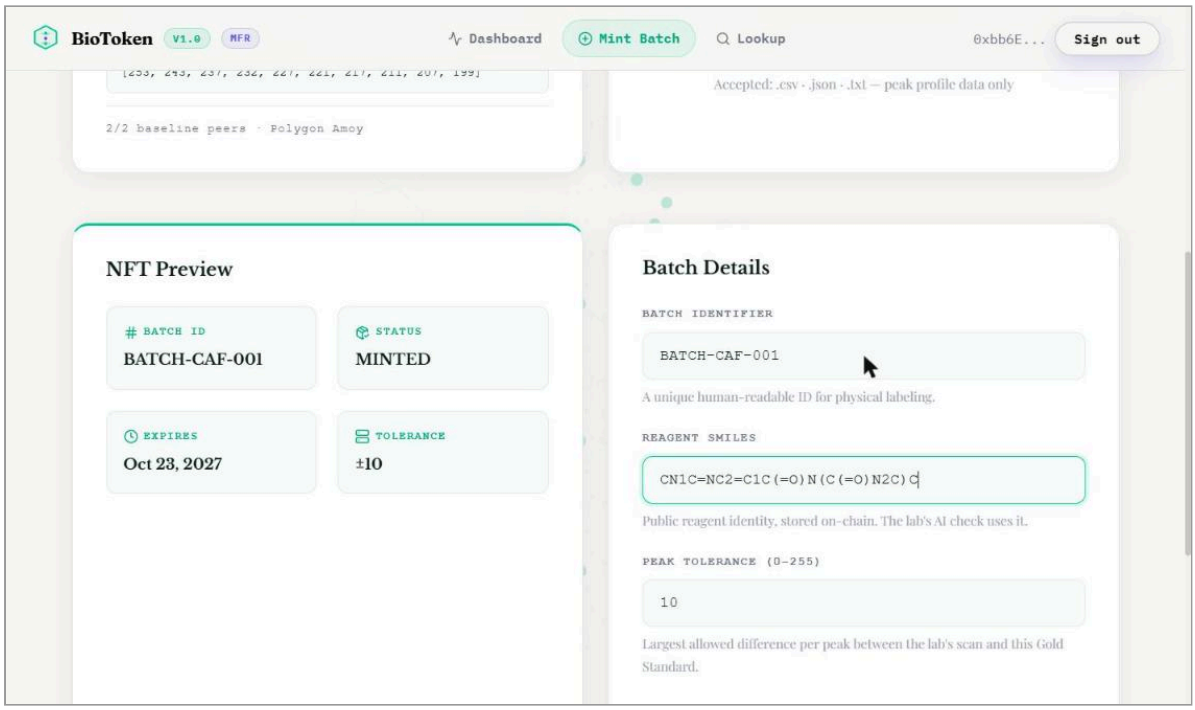


Fig. 16: Batch details entered (BATCH-CAF-001, caffeine SMILES, tolerance 10) and NFT preview

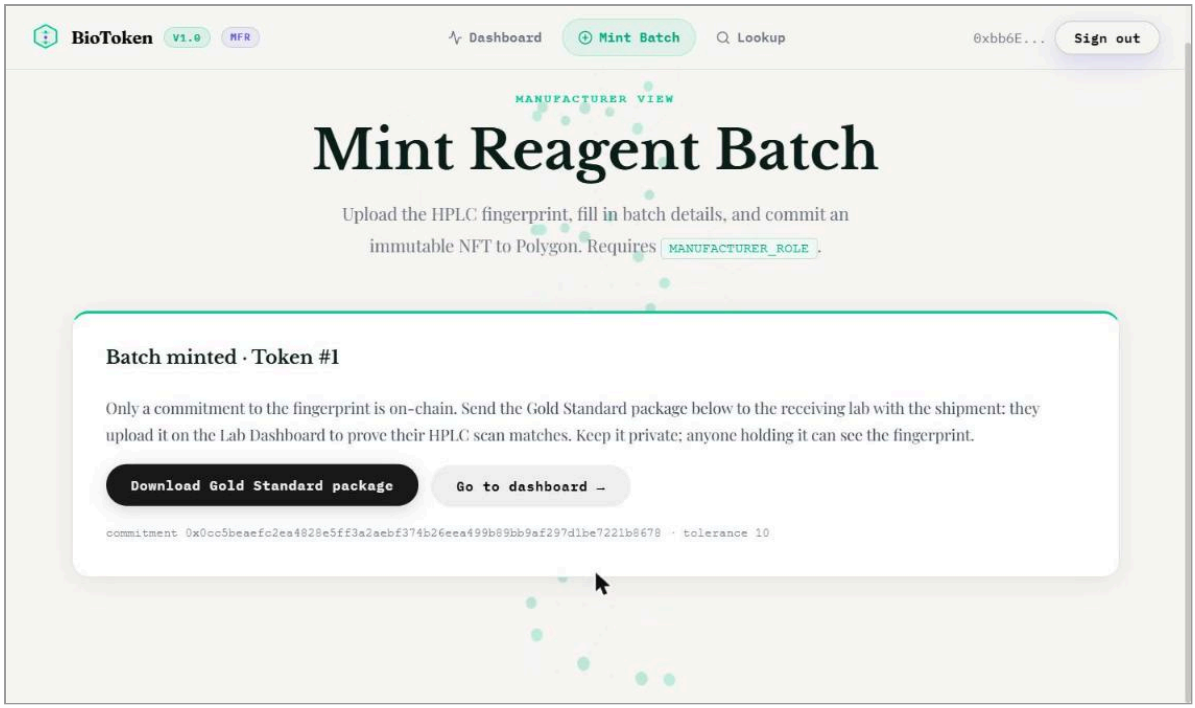


Fig. 17: DApp output after the MetaMask-signed mint: "Batch minted · Token #1" with commitment and tolerance

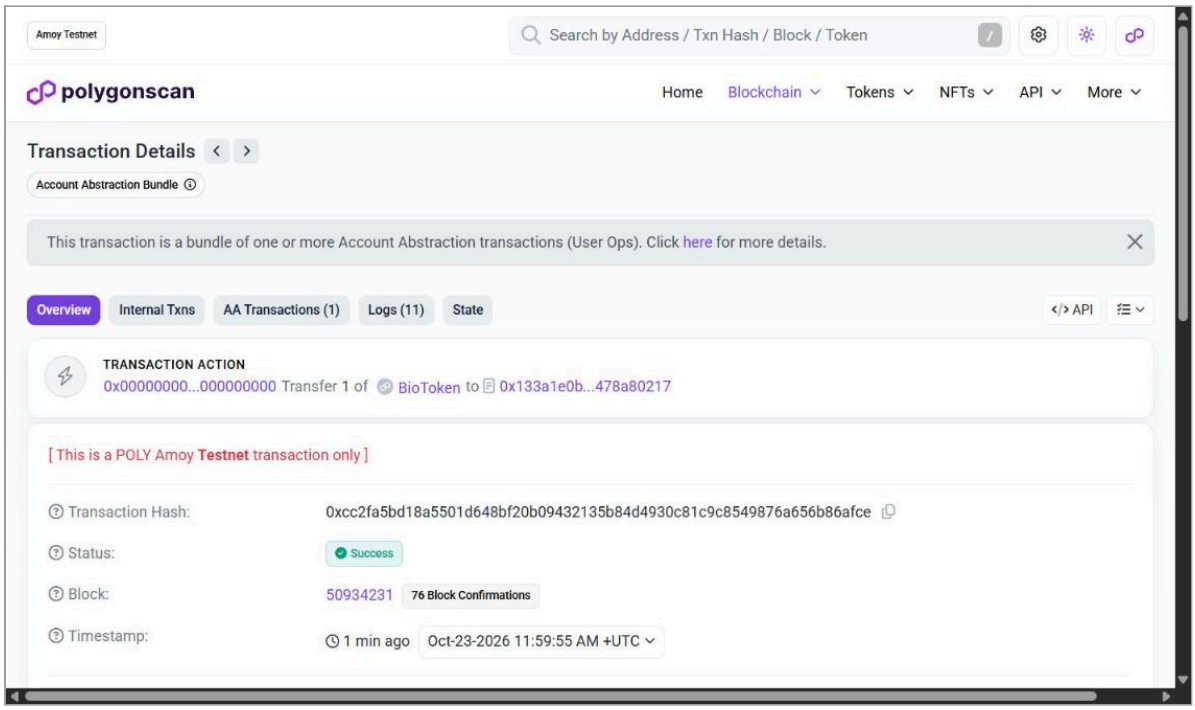


Fig. 18: The DApp mint on PolygonScan: Account Abstraction Bundle, BioToken transferred (minted) to the smart account, block 50934231

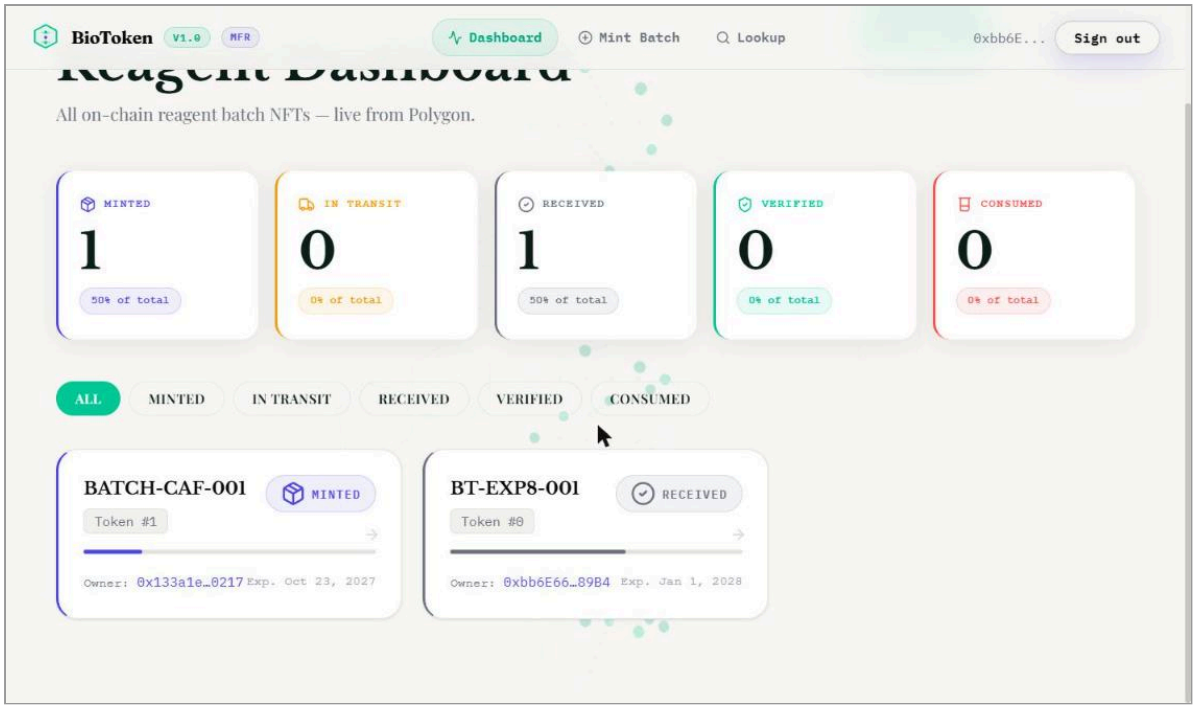


Fig. 19: Dashboard after the DApp mint: Token #1 BATCH-CAF-001 (MINTED, from the DApp) and Token #0 BT-EXP8-001 (RECEIVED, from Remix)

Problems faced and fixes

- **"creation of BioToken errored: error.indexOf is not a function" in Remix.** Remix 2.6.8 hides the real RPC error. Logging the wallet requests showed two causes. First, the MetaMask RPC endpoint for Amoy was failing (MetaMask error `-32002 RPC endpoint not found or unavailable`), fixed by setting the Amoy RPC to `polygon-amoy-bor-rpc.publicnode.com`. Second, the network rejected some transactions with "gas tip cap 1.5 gwei, minimum needed 25 gwei".
- **Minimum tip on Amoy.** MetaMask could not get a fee suggestion (`eth_maxPriorityFeePerGas` is not available through MetaMask) and fell back to a 1.5 gwei tip. The fix was a 30 gwei priority fee (60 gwei max fee), set in MetaMask's Advanced fee screen or suggested by the page with the transaction.
- **Privy "Initializing wallet…".** An e-mail login was waiting for an embedded wallet. The local session was cleared and the user logged in with **Continue with a wallet → MetaMask**, the method this experiment requires.

Program:

1. Smart contract (BioToken.sol – lifecycle functions used in this experiment)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
import "@openzeppelin/contracts/utils/cryptography/MessageHashUtils.sol";

contract BioToken is ERC721, AccessControl {
    bytes32 public constant MANUFACTURER_ROLE = keccak256("MANUFACTURER_ROLE");
    bytes32 public constant LAB_ROLE          = keccak256("LAB_ROLE");
    bytes32 public constant AI_ORACLE_ROLE    = keccak256("AI_ORACLE_ROLE");
```

```
    enum TokenStatus { MINTED, IN_TRANSIT, RECEIVED, VERIFIED, CONSUMED }

    struct TokenData {
        string batchId;  string reagentSmiles;  uint256 expiry;
        bytes32 fingerprintCommitment;  uint8 tolerance;
        TokenStatus status;  bytes marketplaceSig;
    }

    IHplcVerifier public immutable verifier;
    uint256 private _nextTokenId;
    mapping(uint256 => TokenData) private _tokenData;

    event TokenMinted(uint256 indexed tokenId, string batchId, address indexed manufacturer);
    event CustodyTransferred(uint256 indexed tokenId, address indexed from, address indexed to);
    event TokenReceived(uint256 indexed tokenId, address indexed lab);
    event TokenVerified(uint256 indexed tokenId, address indexed lab);
    event TokenConsumed(uint256 indexed tokenId);
    error InvalidStatusTransition(uint256 tokenId, TokenStatus current, TokenStatus expected);

    constructor(address verifierAddress, address aiOracle) ERC721("BioToken", "BIO") {
        verifier = IHplcVerifier(verifierAddress);
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(MANUFACTURER_ROLE, msg.sender);
        _grantRole(AI_ORACLE_ROLE, aiOracle);
    }

    function mintToken(string calldata batchId, string calldata reagentSmiles, uint256 expiry,
                       bytes32 fingerprintCommitment, uint8 tolerance, bytes calldata
marketplaceSig)
        external onlyRole(MANUFACTURER_ROLE) returns (uint256)
    {
        if (fingerprintCommitment == bytes32(0) || uint256(fingerprintCommitment) >=
SNARK_SCALAR_FIELD)
            revert InvalidCommitment();
        if (expiry <= block.timestamp) revert InvalidExpiry();
        uint256 tokenId = _nextTokenId++;
        _safeMint(msg.sender, tokenId);
        _tokenData[tokenId] = TokenData(batchId, reagentSmiles, expiry, fingerprintCommitment,
                                        tolerance, TokenStatus.MINTED, marketplaceSig);
        emit TokenMinted(tokenId, batchId, msg.sender);
        return tokenId;
    }

    function transferCustody(uint256 tokenId, address lab) external {
        _requireStatus(tokenId, TokenStatus.MINTED);
        address holder = ownerOf(tokenId);
        if (!_isAuthorized(holder, msg.sender, tokenId)) revert NotTokenHolder(tokenId,
msg.sender);
        if (!hasRole(LAB_ROLE, lab)) revert RecipientNotLab(lab);
        _tokenData[tokenId].status = TokenStatus.IN_TRANSIT;
        _custodyTransfer = true;  _transfer(holder, lab, tokenId);  _custodyTransfer = false;
        emit CustodyTransferred(tokenId, holder, lab);
    }

    function confirmReceipt(uint256 tokenId) external onlyRole(LAB_ROLE) {
        _requireHolder(tokenId);
        _requireStatus(tokenId, TokenStatus.IN_TRANSIT);
        _tokenData[tokenId].status = TokenStatus.RECEIVED;
        emit TokenReceived(tokenId, msg.sender);
    }

    // verifyProof(tokenId, a, b, c, aiAttestation): AI oracle signature + Groth16 proof ->
VERIFIED
    // consumeToken(tokenId): VERIFIED -> CONSUMED (terminal)

    function getTokenData(uint256 tokenId) external view returns (TokenData memory) {
```

```
        ownerOf(tokenId);  return _tokenData[tokenId];
    }
    function totalMinted() external view returns (uint256) { return _nextTokenId; }
}
```

2. DApp – MetaMask connection through Privy (frontend/src/main.jsx)

```
<PrivyProvider
  appId={import.meta.env.VITE_PRIVY_APP_ID}
  config={{
    loginMethods: ['email', 'google', 'wallet'],     // 'wallet' -> MetaMask
    appearance: { theme: 'light', accentColor: '#10b981' },
    embeddedWallets: { createOnLogin: 'users-without-wallets' },
    defaultChain: polygonAmoy,
    supportedChains: [polygonAmoy],
  }}
>
  <QueryClientProvider client={queryClient}>
    <WagmiProvider config={wagmiConfig}><App /></WagmiProvider>
  </QueryClientProvider>
</PrivyProvider>
```

3. DApp – MetaMask as owner of the gasless smart account (frontend/src/lib/pimlico.js)

```
async function buildSmartAccount(privyWallet) {
  await privyWallet.switchChain(80002)                        // Polygon Amoy
  const eip1193Provider = await privyWallet.getEthereumProvider()   // MetaMask provider
  const walletClient = createWalletClient({ chain: polygonAmoy, transport: custom(eip1193Provider)
})
  return toSimpleSmartAccount({
    client: publicClient, owner: walletClient,               // MetaMask signs UserOperations
    entryPoint: { address: entryPoint07Address, version: '0.7' },
  })
}

export async function createSmartClient(privyWallet) {
  const pimlico = createPimlicoClient({ transport: http(PIMLICO_RPC),
                                        entryPoint: { address: entryPoint07Address, version: '0.7'
} })
  const smartAccount = await buildSmartAccount(privyWallet)
  return createSmartAccountClient({
    account: smartAccount, chain: polygonAmoy,
    bundlerTransport: http(PIMLICO_RPC),
    paymaster: pimlico,                                      // gas sponsored by the paymaster
    paymasterContext: { sponsorshipPolicyId: import.meta.env.VITE_PIMLICO_POLICY_ID },
  })
}
```

4. Direct MetaMask connection with ethers.js (frontend/src/hooks/useContract.js)

```
const browserProvider = new BrowserProvider(window.ethereum);
const network = await browserProvider.getNetwork();
if (Number(network.chainId) !== TARGET_CHAIN_ID) {               // 80002
  try {
    await window.ethereum.request({ method: 'wallet_switchEthereumChain',
                                     params: [{ chainId: CHAIN_OPTIONS[TARGET_CHAIN_ID].chainId }]
});
  } catch (switchError) {
    if (switchError.code === 4902)                               // network not added yet
      await window.ethereum.request({ method: 'wallet_addEthereumChain',
                                       params: [CHAIN_OPTIONS[TARGET_CHAIN_ID]] });
  }
}
await browserProvider.send("eth_requestAccounts", []);           // MetaMask connect pop-up
const signer  = await browserProvider.getSigner();
const contract = new Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);
```

Conclusion:

The MetaMask wallet was configured on the Polygon Amoy testnet and integrated with the BioToken DApp. The BioToken ERC-721 contract was compiled and deployed from Remix through MetaMask, and the deployment was verified on PolygonScan. Its supply-chain lifecycle (role grant, mint, custody transfer, receipt) was then driven from Remix, with every state change confirmed in MetaMask. An invalid transition was rejected by the contract. Finally, the project's own React DApp connected to MetaMask through Privy: the user approved the connection and signed messages, and the account received MANUFACTURER_ROLE. A new reagent batch was minted as a gas-sponsored ERC-4337 UserOperation signed in MetaMask. This shows that a wallet keeps the private key with the user while still authorising every on-chain action in a decentralized application.